

POO 3 (JAVA ET JAVA AVANCÉ)

Prof. Nisrine DAD

4° Ingénierie Informatique et Réseaux - Semestre I

Ecole Marocaine des Sciences d'Ingénieur

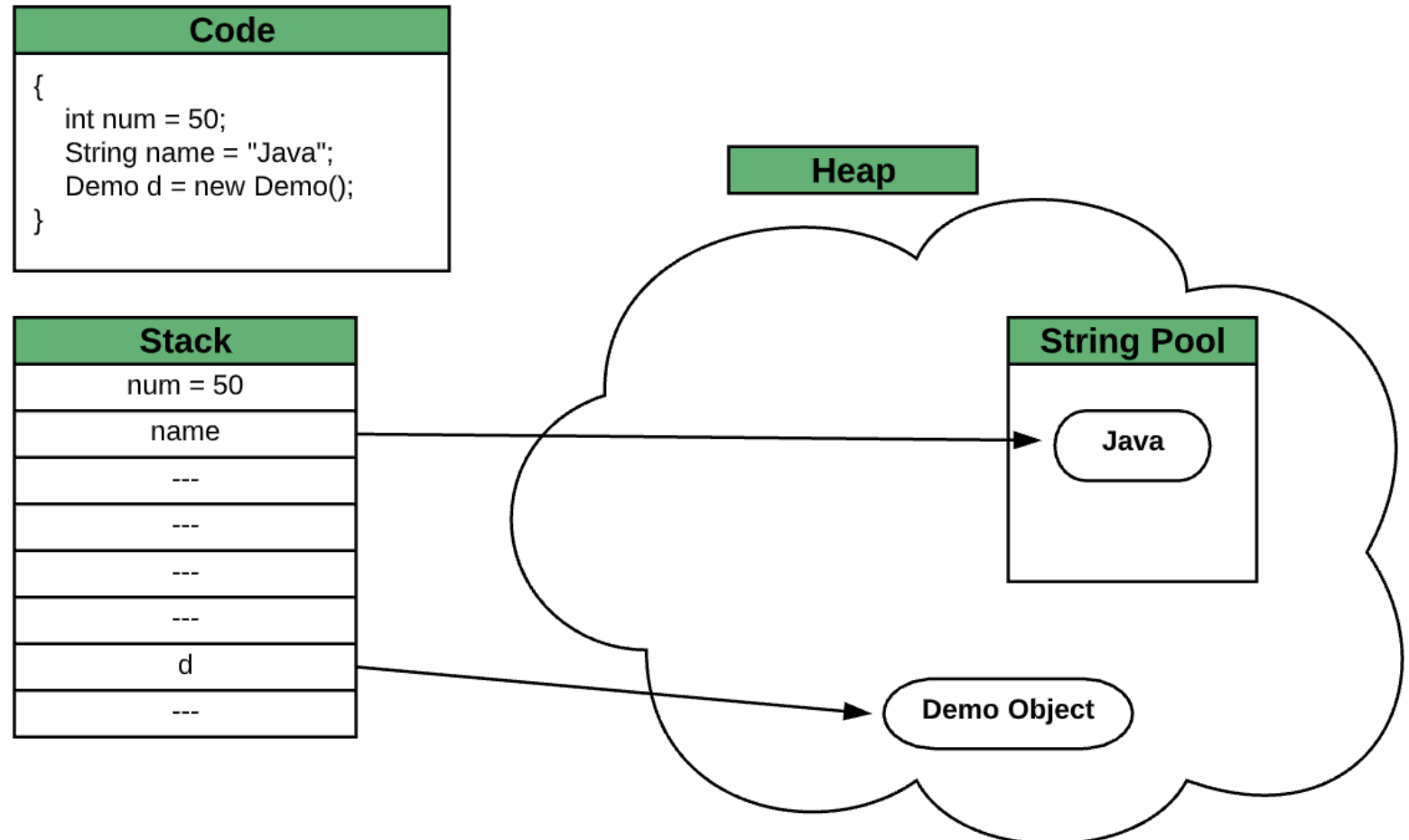
Année Universitaire : 2024/2025

III. CONCEPTS DE LA PROGRAMMATION ORIENTÉE OBJETS (POO) EN JAVA

III. CONCEPTS DE LA POO EN JAVA

Mémoires Stack et Heap

- Mémoire de pile=**Stack**
- Mémoire de tas=**Heap**.
- Le **Stack** contient les primitives, les références et les invocations de méthodes.
- Le **Heap** est utilisée pour stocker des objets.

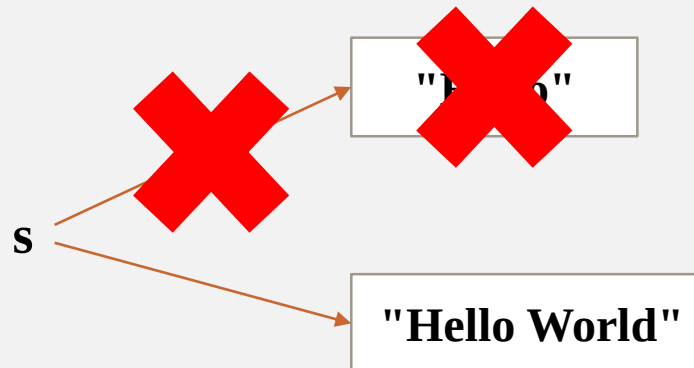


III. CONCEPTS DE LA POO EN JAVA

La classe String

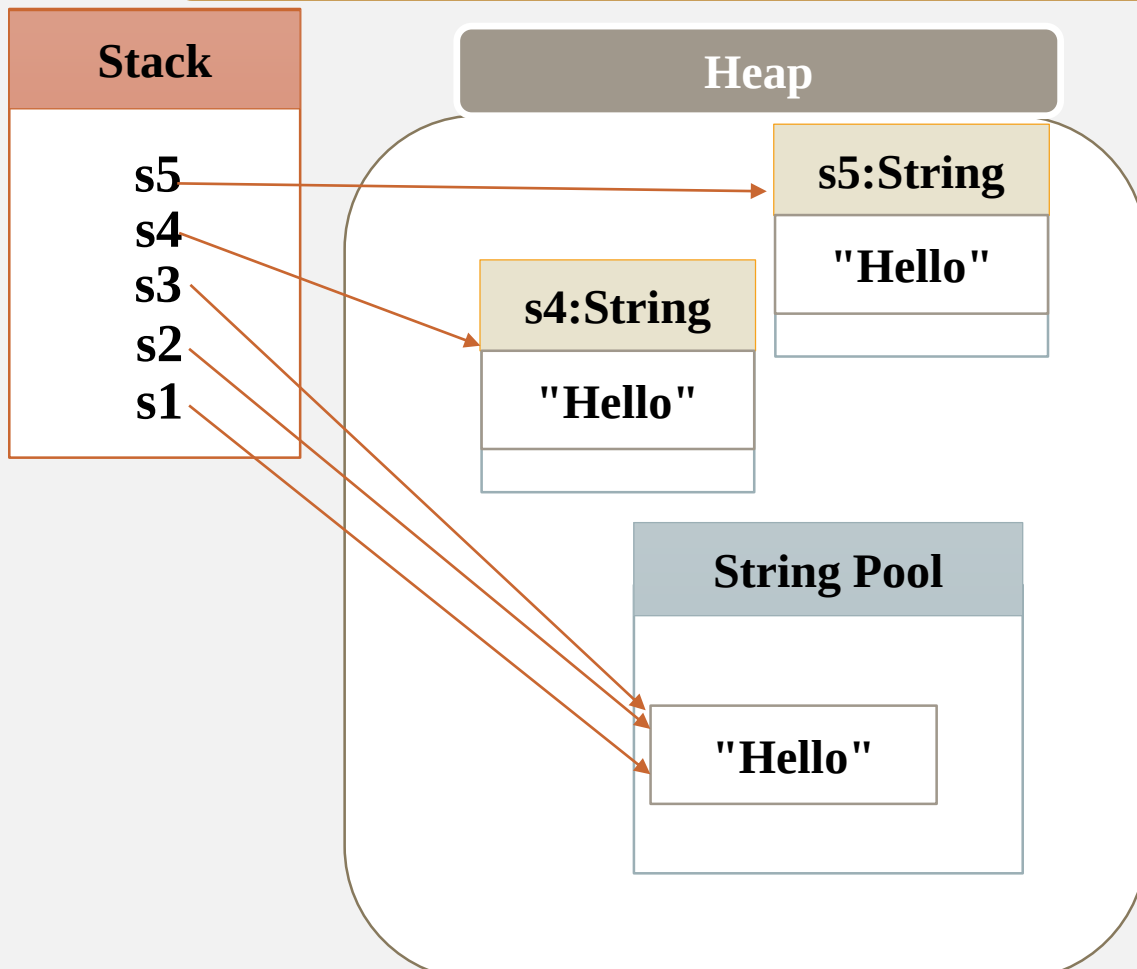
- **Attention** String n'est pas un type de base. Il s'agit d'une classe défini dans l'API Java (package **java.lang**), permettant de manipuler les chaînes de caractères.
- Les objets de type « String » sont **immuables**. En effet, la variable de type référence peut changer de valeur, et pointer vers une nouvelle chaîne créée à un autre emplacement.

```
String s = "Hello";  
s="Hello World";
```



III. CONCEPTS DE LA POO EN JAVA

String littéral vs String objet



```
String s1 = "Hello";  
String s2 = "Hello";  
String s3 = "Hello";
```

```
System.out.println(s2==s1);  
// Pour voir si les 2 variables vers la même  
// chose, on utilise ==  
System.out.println(s2.equals(s1));  
// Pour voir si les 2 variables ont la même  
// valeur, on utilise equals
```

```
String s4=new String("Hello");  
String s5=new String("Hello");
```

```
System.out.println(s4==s5);  
System.out.println(s4.equals(s5));
```

true
true
false
true

III. CONCEPTS DE LA POO EN JAVA

La classe String: Quelques opérations

- Le code suivant montre des exemples des opérations sur les String les plus fréquentes.

```
String s = "Hello"; // Création sans new dans Le String Pool
String str=new String("World"); // Création avec new directement dans Le Heap

System.out.println(s);
System.out.println(str);

System.out.println(s+" "+str); // Concaténation avec +
System.out.println(s.concat(" ").concat(str));
//Concaténation avec .concat(String)
```

```
Hello
World
Hello World
Hello World
```

III. CONCEPTS DE LA POO EN JAVA

La classe String

```
String s = "Hello";  
String ss= "hello";
```

```
System.out.println("La longueur de "+s+" est: "+s.length());  
System.out.println("L'indice de 'lo' dans "+s+" est: "+s.indexOf("lo"));  
System.out.println("Le char dans la position 1 dans "+s+" est: "+s.charAt(1));
```

```
System.out.println(s.equals(ss));  
System.out.println(s.equalsIgnoreCase(ss));
```

```
String chemin="D:/EMSI/cours";  
String sep="/";  
String[] elements=chemin.split(sep);  
for(int i=0;i<elements.length;i++)  
    System.out.print(elements[i]+" ");
```

```
La longueur de Hello est: 5  
L'indice de 'lo' dans Hello est: 3  
Le char dans la position 1 dans Hello est: e  
false  
true  
D: EMSI cours
```

III. CONCEPTS DE LA POO EN JAVA

La classe StringTokenizer

- La classe StringTokenizer nous aide à diviser Strings en plusieurs jetons.

```
String s1= "D:/EMSI/cours";  
StringTokenizer st1=new  
StringTokenizer(s1,"/");  
while(st1.hasMoreTokens()) {  
    System.out.println(st1.nextToken());  
}
```

```
D:  
EMSI  
cours
```


III. CONCEPTS DE LA POO EN JAVA

Exercice

- Quel est le résultat du code suivant?

```
String ch1 = new String();
System.out.println("A - ch1 =:" + ch1 + ":");
String ch2 = "hello";
System.out.println("B - ch2 =:" + ch2 + ":");
String ch3 = new String("bonjour");
System.out.println("C - ch3 =:" + ch3 + ":");
String ch4 = new String(ch3);
System.out.println("D - ch4 =:" + ch4 + ":");
ch3 = "bonsoir";
System.out.println("E - ch4 =:" + ch4 + ": ch3 =:" + ch3 + ":");
ch4 = ch3;
ch3 = "au revoir";
System.out.println("F - ch4 =:" + ch4 + ": ch3 =:" + ch3 + ":");
```

A - ch1 =:
B - ch2 =:hello:
C - ch3 =:bonjour:
D - ch4 =:bonjour:
E - ch4 =:bonjour: ch3 =:bonsoir:
F - ch4 =:bonsoir: ch3 =:au revoir:

III. CONCEPTS DE LA POO EN JAVA

Exercice

- Écrire un programme qui, pour une chaîne de caractères, affiche :
 - un caractère sur deux (le premier étant affiché),
 - le premier et le dernier caractère.

```
String ch = "Hello world";
System.out.print("un caractere sur deux : ");
for (int i = 0; i < ch.length(); i += 2)
System.out.print(ch.charAt(i));
System.out.println();
System.out.println("Premier caractere = " + ch.charAt(0));
System.out.println("Dernier caractere = " + ch.charAt(ch.length() - 1));
```

III. CONCEPTS DE LA POO EN JAVA

Exercice

- Écrire un programme qui prend une chaîne de caractères et l'affiche verticalement comme dans cet exemple :

La chaine de caractère originale: 785412

Son affichage:

7

8

5

4

1

2

```
System.out.println("La chaine de caractère originale: "+ch+ "\nSon affichage:");  
for (int i=0 ; i<ch.length() ; i++)  
    System.out.println (ch.charAt(i)) ;
```

III. CONCEPTS DE LA POO EN JAVA

Exercice

- Écrire un programme qui prend un String et qui indique combien de fois sont présentes chacune des voyelles a, e, i, o, u ou y, que celles-ci soient écrites en majuscules ou en minuscules, comme dans cet exemple :

```
Le texte est : HELLO World
il comporte :
0 fois la lettre a
1 fois la lettre e
0 fois la lettre i
2 fois la lettre o
0 fois la lettre u
0 fois la lettre y
```

```
char voy[] = { 'a', 'e', 'i', 'o', 'u', 'y' };
int nVoy[] = new int[voy.length];
for (int i = 0; i < nVoy.length; i++)
    nVoy[i] = 0;
String mot = "HELLO World";
System.out.println("Le texte est : "+mot);
mot = mot.toLowerCase();

for (int i = 0; i < mot.length(); i++)
    for (int j = 0; j < voy.length; j++)
        if (mot.charAt(i) == voy[j])
            nVoy[j]++;

System.out.println("il comporte : ");

for (int i = 0; i < voy.length; i++)
    System.out.println(nVoy[i] + " fois la
    lettre " + voy[i]);
```

III. CONCEPTS DE LA POO EN JAVA

La classe **String**

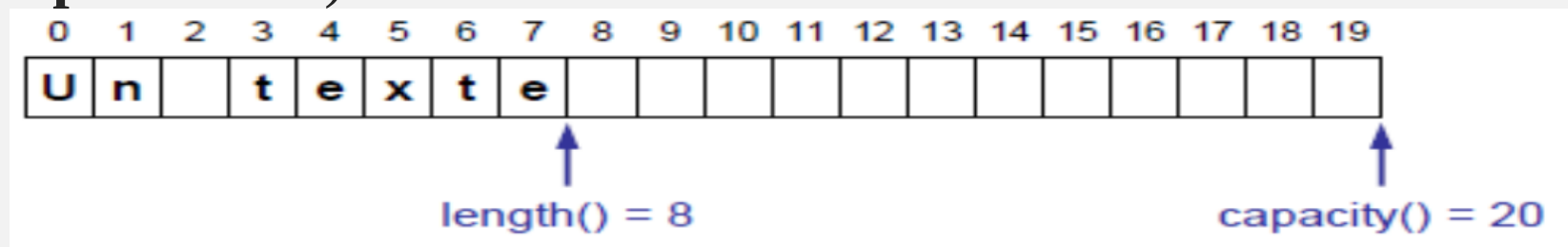
- **L'avantage** des objets immuables est qu'ils peuvent être **partagés sans risque**.
- Mais si on utilise fréquemment ce type d'objets à l'intérieur des boucles, par exemple, on peut avoir la création de plusieurs d'objets (temporaires) avec un coût non négligeable (mémoires, temps d'exécution).
- Dans ce cas, il est préférable de déclarer et utiliser des objets modifiables qui sont de type **StringBuffer** ou **StringBuilder**.

III. CONCEPTS DE LA POO EN JAVA

La classe **StringBuffer**

- Contrairement à la classe `String`, la classe **StringBuffer** permet de créer des **chaînes de caractères modifiables** (la taille et le contenu peuvent varier durant l'exécution de l'application).
- La **notion de capacité** représente la taille du buffer interne qui mémorise la chaîne de caractères (**nombre total de caractères disponibles avant de devoir agrandir la taille du buffer interne**). La **capacité** est automatiquement augmentée lorsque c'est nécessaire.
- La **capacité initiale** peut être définie lors de la création d'un objet `StringBuffer` (paramètre du constructeur, **16 par défaut**).

Ne pas confondre la **capacité** et la **longueur** d'un objet `StringBuffer`.



III. CONCEPTS DE LA POO EN JAVA

La classe StringBuffer

- Pour créer un objet de type StringBuffer, on doit utiliser l'opérateur **new** (...), car il n'y a pas de syntaxe simplifiée.
- On peut aussi **définir une capacité initial** lors de la création.

```
21
5
2000
11
Hello World
```

```
StringBuffer sb1=new StringBuffer("Hello");
System.out.println(sb1.capacity());
System.out.println(sb1.length());
```

```
StringBuffer sb2=new StringBuffer(2000);
sb2.append(sb1);
sb2.append(" World");
System.out.println(sb2.capacity());
System.out.println(sb2.length());
```

```
System.out.println(sb2);
```

III. CONCEPTS DE LA POO EN JAVA

La classe **StringBuffer**: Quelques opérations

<code>append()</code>	Ajoute un élément à la fin de la chaîne de caractères (surcharge pour différents types d'éléments)
<code>insert()</code>	Insère un élément à une position donnée de la chaîne de caractères (surcharge pour différents types d'éléments)
<code>replace()</code>	Remplace une partie d'une chaîne de caractères par une autre
<code>delete()</code>	Efface une portion donnée d'une chaîne de caractères
<code>setCharAt()</code>	Remplace un caractère à une position donnée
<code>deleteCharAt()</code>	Efface (supprime) un caractère à une position donnée
<code>substring()</code>	Extrait une sous-chaîne
<code>toString()</code>	Convertit le contenu en type String

III. CONCEPTS DE LA POO EN JAVA

String vs StringBuilder vs StringBuffer

```
/** Strings are constant; their values cannot be changed after they
    are created. String buffers support mutable strings. */
public final class String
    implements java.io.Serializable, Comparable<String>, CharSequence,
               Constable, ConstantDesc
```

```
/** A mutable sequence of characters. */
public final class StringBuilder
    extends AbstractStringBuilder
    implements java.io.Serializable, Comparable<StringBuilder>, CharSequence
```

```
/**A thread-safe, mutable sequence of characters.*/
public final class StringBuffer
    extends AbstractStringBuilder
    implements Serializable, Comparable<StringBuffer>, CharSequence
```

III. CONCEPTS DE LA POO EN JAVA

Exercice

- Réaliser une classe **Repertoire** permettant de gérer un répertoire téléphonique associant un numéro de téléphone (chaîne de caractères) à un nom. Pour faciliter les choses, on prévoira une classe **Abonne** destinée à représenter un abonné et disposant des fonctionnalités indispensables.
- La classe **Repertoire** devra disposer des fonctionnalités suivantes :
 - **constructeur** recevant un argument de type entier précisant le nombre maximum d'abonnés que pourra contenir le répertoire (cette particularité évite d'avoir à se soucier d'une gestion dynamique du répertoire),
 - méthode **addAbonne** permettant d'ajouter un nouvel abonné ; elle renverra la valeur false si le répertoire est plein, la valeur true sinon,
 - méthode **getNumero** fournissant le numéro associé à un nom d'abonné fourni en argument,
 - méthode **getNAbonnes** qui fournit le nombre d'abonnés figurant dans le répertoire,
 - méthode **getAbonne** fournissant l'abonné dont le rang est fourni en argument,
 - méthode **getAbonnesTries** fournissant un tableau des références des différents abonnés, rangés par ordre alphabétique (pour simplifier, on supposera que les noms sont écrits en minuscules, sans caractères accentués).
- Écrire un petit programme de test.

III. CONCEPTS DE LA POO EN JAVA

Exercice

```
class Abonne {  
    private String nom, numero;  
  
    public Abonne(String nom, String numero) {  
        this.nom = nom;  
        this.numero = numero;}  
  
    public String getNom() {  
        return nom;}  
  
    public String getNumero() {  
        return numero;}  
}
```

```
class Repertoire {

    private int nMax, nAbon;
    private Abonne[] rep;

    public Repertoire(int nMax) {
        this.nMax = nMax;
        rep = new Abonne[nMax];
        nAbon = 0;}

    public boolean addAbonne(Abonne a) {
        if (nAbon >= nMax)
            return false;
        rep[nAbon] = a;
        nAbon++;
        return true;}

    public int getNAbonnes() {
        return nAbon;}

    public Abonne getAbonne(int num) {
        if (num < nAbon)
            return rep[num];
        return null;}
```

```
    public String getNumero(String nom) {
        for (int i = 0; i <= nAbon; i++)
            if (nom.equals(rep[i].getNom()))
                return rep[i].getNumero();
        return null;}

    public Abonne[] getAbonnesTries() {
        Abonne[] repTrie = new Abonne[nAbon];
        for (int i = 0; i < nAbon; i++)
            repTrie[i] = rep[i];
        for (int i = 0; i < nAbon - 1; i++)
            for (int j = i + 1; j < nAbon; j++)
                if ((repTrie[i].getNom()).compareTo
                    (repTrie[j].getNom()) > 0) {
                    Abonne temp = repTrie[i];
                    repTrie[i] = repTrie[j];
                    repTrie[j] = temp;}
        return repTrie;}

}
```

```

Repertoire rep = new Repertoire(10);
System.out.println("il y a " + rep.getNAbonnes() + "
abonnes");
Abonne a = new Abonne("Amini", "06 11 22 33 44");
Abonne b = new Abonne("Alami", "07 11 11 33 44");
rep.addAbonne(a);
rep.addAbonne(b);
rep.addAbonne(new Abonne("Mazin", "06 66 55 44 33"));
rep.addAbonne(new Abonne("Kamili", "05 35 23 45 67"));
System.out.println("il y a " + rep.getNAbonnes() + "
abonnes");

System.out.println("qui sont : ");
for (int i = 0; i < rep.getNAbonnes(); i++)
    System.out.println(rep.getAbonne(i).getNom() + " "
+ rep.getAbonne(i).getNumero());

System.out.println("ou encore, par ordre alphabetique");
Abonne[] abonnes = rep.getAbonnesTries();
for (int i = 0; i < abonnes.length; i++)
    System.out.println(abonnes[i].getNom() + " " +
abonnes[i].getNumero());

```

```

il y a 0 abonnes
il y a 4 abonnes
qui sont :
Amini 06 11 22 33 44
Alami 07 11 11 33 44
Mazin 06 66 55 44 33
Kamili 05 35 23 45 67
ou encore, par ordre alphabetique
Alami 07 11 11 33 44
Amini 06 11 22 33 44
Kamili 05 35 23 45 67
Mazin 06 66 55 44 33

```